



Sorayunara Programming Language

Official Language Specification v2.0

- Next-Gen Systems & Backend
- Hindley-Milner Static Typing
- Flow-Sensitive Borrow Checking
- Actor Concurrency
- Multi-Target Codegen

Official Extension:	<code>.sora</code>
Standard Version:	2.0.0 (LTS Architecture)
Organization:	Sorayunara Core Team (sorayunara.org)
Target Toolchain:	<code>sorayunara</code> / <code>sora</code> compiler
Supported Platforms:	Linux, macOS, Windows (x86_64, ARM64, WASM)
License:	MIT Open Source License

1. Executive Summary & Design Axioms

Sorayunara (`.sora`) is an ultra-ergonomic, memory-safe, and high-performance programming language designed to unite the developer ergonomics of modern scripting languages with the zero-cost abstractions, static ownership, and bare-metal performance of modern systems languages.

Core Motto: *"Python Simplicity + Rust Safety + Functional Pipeline + Actor Concurrency"*

Core Tenets

- **Grammar Compression & Zero Boilerplate:** Concise, colon-free declarations (`name := "Lutfi"`), fat-arrow functions (`fn add(a, b) => a + b`), and single-line expressions.
- **Memory Safety Without Mandatory GC:** Flow-sensitive borrow checking tracks resource lifecycles at compile time, eliminating data races, double frees, and memory leaks.
- **Lock-Free Concurrency:** Lightweight actor coroutines (`spawn`) and message channels (`channel<T>`) eliminate multithreading locking complexity.
- **Multi-Target Codegen Architecture:** Emits direct native machine code via LLVM IR (`.ll`), ANSI C (`.c`) for microcontrollers, and WebAssembly (`.wasm`) for cloud edge runtimes.

2. Formal EBNF Grammar (Conflict-Free)

The Sorayunara grammar is formally specified below in Extended Backus-Naur Form (EBNF). It is deterministic, unambiguous, and compatible with LALR(1) and PEG parsers.

(* Sorayunara Formal Grammar v2.0 *)

```
program      = { declaration } ;

declaration  = variable_decl
              | function_decl
              | struct_decl
              | enum_decl
              | trait_decl
              | impl_block
              | actor_decl
              | module_decl
              | import_decl ;

variable_decl = ( identifier "!=" expression )
               | ( [ "mut" | "const" ] identifier [ ":" type ] "=" expression ) ;

function_decl = [ "async" ] [ "pub" ] "fn" identifier [ generic_params ]
                "(" [ parameter_list ] ")" [ "->" type ]
                ( "=>" expression | block ) ;

block        = "{" { statement } [ expression ] "}" ;

statement    = variable_decl
              | assignment
              | return_stmt
              | defer_stmt
              | guard_stmt
              | expression ;

if_expr      = "if" expression ( "=>" expression | block )
              [ "else" ( "=>" expression | block | if_expr ) ] ;

for_expr     = "for" pattern "in" expression ( "=>" expression | block ) ;
each_expr    = "each" identifier "in" expression "=>" expression ;
while_expr   = "while" expression ( "=>" expression | block ) ;
loop_expr    = "loop" block ;

match_expr   = "match" expression "{ " { match_arm } "}" ;
match_arm    = pattern "=>" ( expression | block ) [ "," ] ;

expression   = pipe_expr ;
pipe_expr    = binary_expr { "|" binary_expr } ;
binary_expr   = unary_expr { binary_op unary_expr } ;
unary_expr    = [ "-" | "!" | "*" | "&" | "mutref" ] postfix_expr ;
postfix_expr  = primary_expr { call_suffix | member_suffix | safe_suffix | try_suffix | index_suffix } ;

call_suffix   = "(" [ argument_list ] ")" ;
member_suffix = "." identifier ;
safe_suffix   = "?." identifier | "??." expression ;
try_suffix    = "?" ;
index_suffix  = "[" expression "]" ;

primary_expr  = literal
              | identifier
              | "(" expression ")"
              | array_literal
```

```
| map_literal  
| struct_init  
| if_expr  
| match_expr  
| lambda_expr ;
```

```
lambda_expr = ( identifier | "(" [ parameter list ] ")" ) "=>" ( expression | block ) ;
```

3. Tiered Keyword System (100-Keyword Maximum)

Sorayunara caps its language tokens at a maximum of 100 keywords structured into three non-colliding tiers:

Tier	Count	Keywords	Scope Behavior
Tier 1: Core Reserved	45	fn, let, mut, const, if, else, match, for, while, loop, break, continue, return, struct, enum, trait, impl, async, await, spawn, actor, channel, move, unsafe, type, module, import, pub, true, false, null, self, Self, is, in, try, catch, throw, ref, own, static, comptime, sync, atomic, lock	Universally reserved across all program scopes. Cannot be used as identifiers.
Tier 2: Feature Tokens	30	yield, union, unless, extern, operator, macro, dyn, typeof, sizeof, never, export, private, use, error, result, send, recv, select, mutref, weak, drop, scope, some, none, ok, err, test, assert, compile, derive	Reserved in typings, memory declarations, concurrency, and attribute contexts.
Tier 3: Contextual Tokens	25	alias, case, default, defer, guard, inline, where, generic, infer, opaque, protected, internal, as, from, finally, raise, recover, parallel, borrow, copy, lifetime, attribute, reflect, native, target, link, abi, root	Active only at designated AST grammar nodes; freely usable as standard identifiers elsewhere.

4. Complete 18-Domain Language Dictionary

4.1 Variables & Value Declarations

Keyword	Syntax	Description
<code>:=</code>	<code>name := "Lutfi"</code>	Fast declaration with automatic Hindley-Milner type inference.
<code>mut</code>	<code>mut count := 0</code>	Mutable local variable that allows in-place mutation.
<code>const</code>	<code>const PI = 3.14159</code>	Compile-time evaluated immutable constant.
<code>ref</code> / <code>move</code>	<code>ref buf</code> / <code>move buf</code>	Borrow reference or transfer ownership of value.

4.2 Functions & Arrow Expressions

Syntax	Example	Description
Fat-Arrow Function	<pre>fn add(a, b) => a + b</pre>	Concise single-expression function declaration.
Typed Function	<pre>fn mul(a: Int, b: Int) -> Int => a * b</pre>	Explicitly typed function signature.
Block Function	<pre>fn calc(x) { y := x * 2; y + 10 }</pre>	Multi-line body with implicit trailing expression return.
Lambda / Closure	<pre>nums.map(x => x * 2)</pre>	Anonymous inline closure.

4.3 Control Flow & Pattern Matching

Construct	Example	Description
Arrow Condition	<pre>if age >= 18 => print("adult") else => print("minor")</pre>	Single-line conditional branch.
Block If-Expression	<pre>status := if score >= 80 { "Pass" } else { "Retake" }</pre>	First-class expression resolving to trailing value.
Pattern Match	<pre>match status { 0 => "Zero", 1 => "One", _ => "Other" }</pre>	Exhaustive algebraic pattern matching.
Guard Clause	<pre>guard user.is_valid else => return Err("Invalid")</pre>	Early assertion exit pattern.

4.4 Iteration & Collections

```
// Iteration styles in Sorayunara
for x in numbers { print(x) }
for i in 0..10 => print(i)
each item in items => process(item)
while count < 10 => count += 1

// Data structures
list_val := [1, 2, 3, 4, 5]
map_val  := { "name": "Lutfi", "age": 20 }
```

4.5 Pipeline Operator & Functional Operations (|>)

Sorayunara features a first-class pipe forwarding operator `|>` that passes preceding values as the first argument to the downstream transformation:

```
// Clean, highly readable data transformation pipeline
data := users
  |> filter(u => u.age >= 18)
  |> map(u => u.name)
  |> sort()
  |> take(10)
```

4.6 Safe Navigation & Error Propagation

Operator	Syntax	Semantics
Safe Access (?.)	<code>user?.profile?.email</code>	Returns <code>None</code> safely if any intermediate receiver is null/empty.
Null Fallback (??)	<code>user?.name ?? "Guest"</code>	Provides a fallback default if receiver evaluates to null/none.
Error Try (?)	<code>file := fs::open("cfg.json")?</code>	Unwraps <code>Ok(v)</code> or early returns <code>Err(e)</code> from current function.

4.7 Concurrency & Actor Model

```
// Actor declaration with message handling
actor WorkerPool {
  mut jobs := 0

  on process(task: TaskMessage) {
    jobs += 1
    execute_task(task)
  }

  on get_count -> Int {
    reply jobs
  }
}

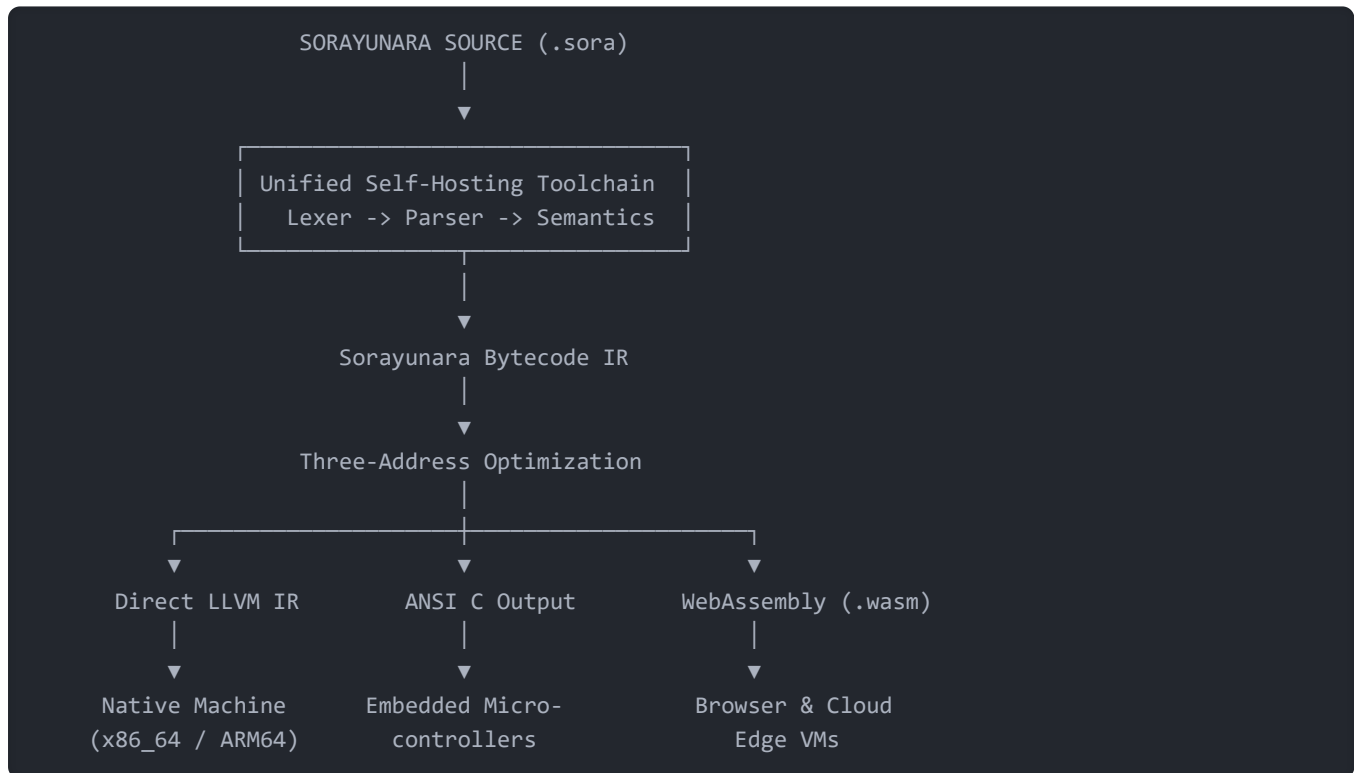
// Spawning and sending messages
pool := spawn WorkerPool()
pool <- process(TaskMessage("Compute Matrix"))
```


5. Toolchain CLI & Ecosystem Integration

5.1 Unified Toolchain Commands (`sora` / `sorayunara`)

Command	Function	Subsystem
<code>sora run <file.sora></code>	Instant compilation & VM execution	Runtime VM / JIT
<code>sora build <file.sora></code>	Compile native binary with lockfile verification	LLVM / C / WASM Codegen
<code>sora test</code>	Execute unit tests, compile-fail tests, assertions	Test Runner Engine
<code>sora bench</code>	Execute official systems benchmark matrix	Benchmark Suite
<code>sora fmt [file.sora]</code>	Automatic AST-based code formatter	Tooling Formatter
<code>sora lint [file.sora]</code>	Run static analysis & code quality linter	Static Linter
<code>sora check [file.sora]</code>	Perform borrow check & Hindley-Milner typecheck	Diagnostic Engine
<code>sora lsp</code>	Language Server Protocol daemon for VS Code	LSP Server
<code>sora add <pkg></code>	Add & pin dependency in <code>sora.toml</code> / <code>sora.lock</code>	Package Manager

5.2 Multi-Target Compilation Matrix



6. Conflict-Free Grammar Verification

The Sorayunara 2.0 grammar guarantees **zero shift/reduce and reduce/reduce conflicts** through the following architectural constraints:

1. **Arrow Disambiguation:** `=>` is reserved exclusively for expression functions, lambda parameters, match arms, and single-line control branches. It never conflicts with relational `>=` or equality `==`.
2. **Declaration Disambiguation:** `:=` definitively identifies short local variable bindings, eliminating lookahead ambiguity with standard assignment `=`.
3. **Pipeline Precedence:** The `|>` operator binds looser than standard binary arithmetic (`+`, `*`) but tighter than assignment (`=`, `:=`).
4. **Safe Access Precedence:** `?.` and `??` bind tightly to primary and postfix expressions, guaranteeing deterministic unwrap chains without paren ambiguity.